Netflix-Style Landing Page

CI/CD with Jenkins on AWS EC2

Author: Ritesh Thorat
Stack: GitHub · Jenkins (Freestyle) · AWS EC2 · Docker · Docker Hub · Nginx

1. Introduction

This portfolio project demonstrates an end-to-end CI/CD workflow for a Netflix-inspired static landing page. GitHub stores the application source. A Jenkins Freestyle job on an AWS EC2 instance builds and tests a Docker image. Docker Hub stores the published image. Docker runs the deployed website on port **8081**.

This is an educational demo with an original streaming-inspired design. It is not the Netflix application and does not use Netflix proprietary source code.

1.1 Project links

Resource	Link
GitHub repository	ritesh-thorat-netflix-cicd
Docker account (billing page)	Docker account
LinkedIn	Ritesh Thorat
AWS Builder Center	@tu13dekh

> **Docker Hub namespace note:** The provided Docker link is an account **billing page**. It is not a public Docker Hub profile or repository. This guide uses the placeholder `<DOCKERHUB_USER>` for the Docker Hub username that owns the image repository. Replace it only after you confirm the real username. Do not assume the identifier in the billing URL is the Docker Hub namespace.

1.2 Placeholders used in this guide

Placeholder	Meaning
`YOUR_EC2_PUBLIC_IP`	Public IPv4 address of your EC2 instance, shown in the AWS console
`<DOCKERHUB_USER>`	Your confirmed Docker Hub username (the image namespace)
`your-key.pem`	Filename of the EC2 key pair you downloaded when launching the instance

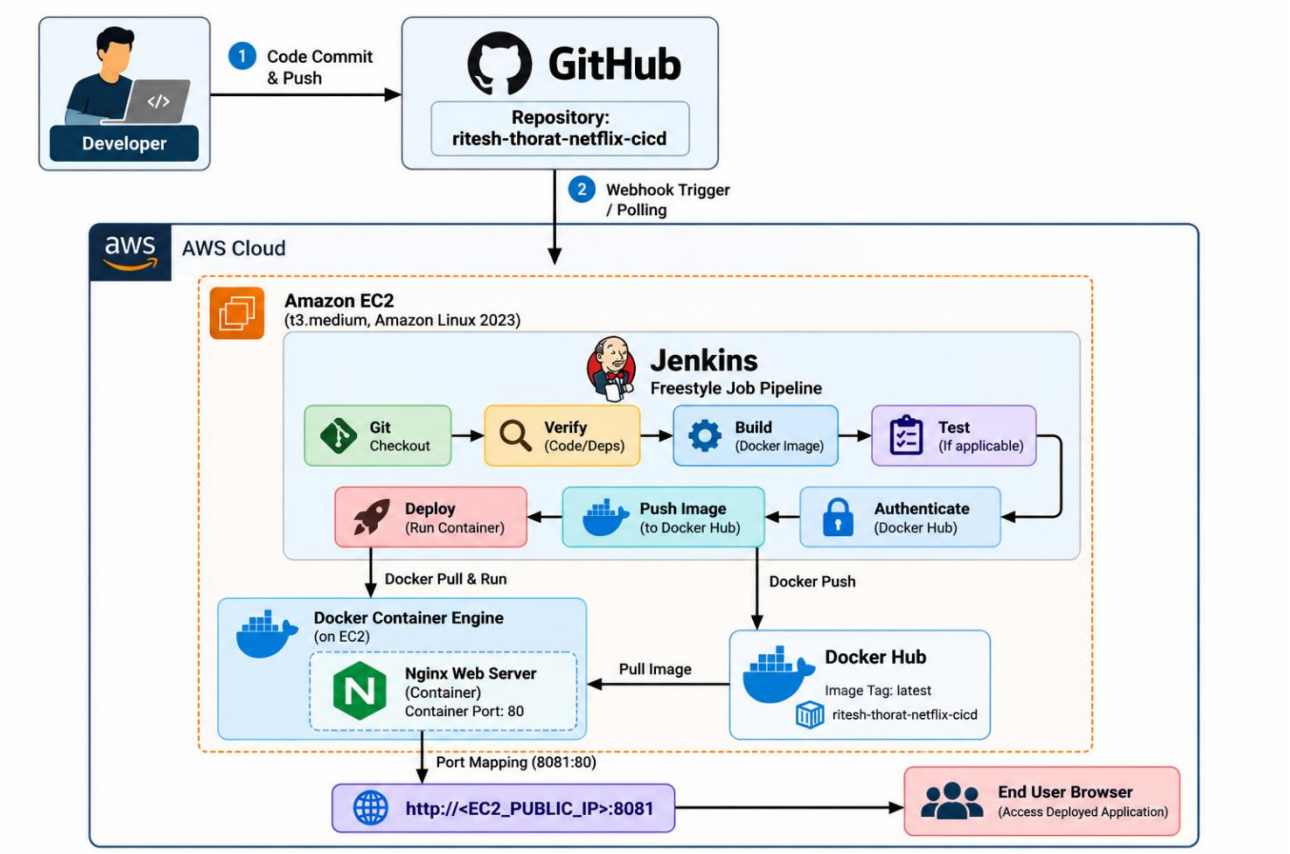
2. Prerequisites

- An AWS account with permission to launch EC2 instances and edit security groups
- An EC2 key pair (`.pem` file) for SSH access
- A GitHub account with access to the project repository
- A Docker Hub account with a repository named `ritesh-thorat-netflix-cicd` (see Phase 5)
- Your current public IP address, for restricting security group rules

- Basic familiarity with a Linux terminal

3. Architecture

The application source flows through this pipeline:



The Jenkins controller and the deployed Docker container run on the same EC2 instance. Jenkins builds the image locally, pushes it to Docker Hub, and then starts the container from that same local image.

Port	Service
22	SSH administration
8080	Jenkins web interface
8081	Deployed website

Restrict all three inbound rules to your own public IP. Do not expose SSH or Jenkins to the entire internet.

4. Setup Phases

4.1 Phase 1 — Launch and secure the EC2 instance

Launch an EC2 instance with this lab configuration:

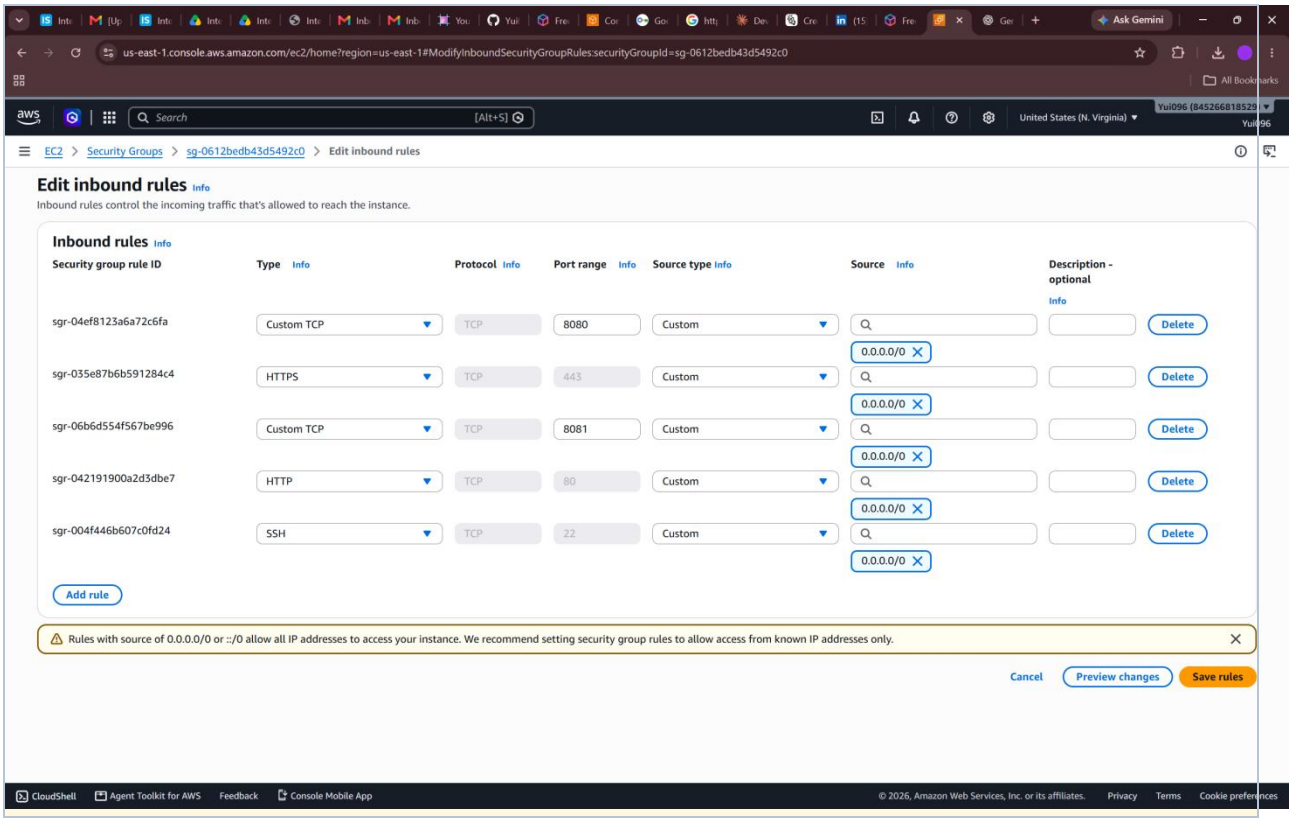
Setting	Value
Operating system	Amazon Linux 2023

Setting	Value
Instance type	t3.medium
Storage	20–30 GiB recommended
Network	VPC/subnet with outbound internet access
Security group	TCP 22, 8080, and 8081 from your public IP

Add these inbound security group rules:

Type	Protocol	Port	Source	Purpose
SSH	TCP	22	My IP	EC2 administration
Custom TCP	TCP	8080	My IP	Jenkins web interface
Custom TCP	TCP	8081	My IP	Deployed website

Wait until the instance is **Running** and has passed its status checks. Record its public IPv4 address.



Connect with EC2 Instance Connect or SSH:

```
ssh -i your-key.pem ec2-user@YOUR_EC2_PUBLIC_IP
```

Replace `your-key.pem` with your key filename and `YOUR_EC2_PUBLIC_IP` with your instance's public IPv4 address.

4.2 Phase 2 — Install Java and Jenkins

Update the system and install Amazon Corretto 21:

```
sudo dnf update -y
sudo dnf install -y java-21-amazon-corretto
java -version
```

Add the Jenkins stable repository, import its signing key, and install Jenkins:

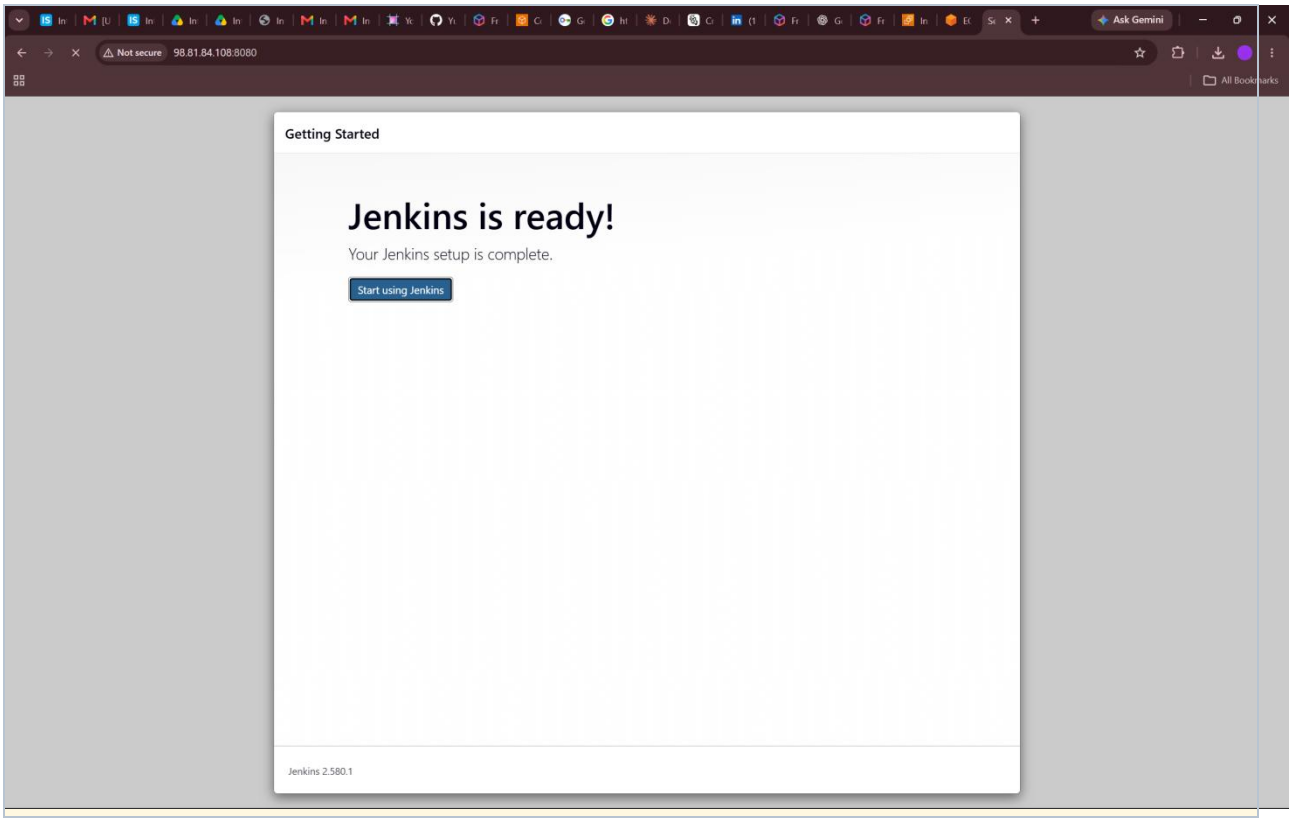
```
sudo wget -O /etc/yum.repos.d/jenkins.repo \
  https://pkg.jenkins.io/rpm-stable/jenkins.repo
sudo rpm --import \
  https://pkg.jenkins.io/rpm-stable/jenkins.io-2026.key
sudo dnf install -y jenkins
sudo systemctl enable --now jenkins
sudo systemctl status jenkins
```

> **Assumption:** Jenkins rotates its repository signing key periodically. If the `rpm --import` command fails, check the current key filename on the official Jenkins Red Hat installation page.

Open `http://YOUR_EC2_PUBLIC_IP:8080` in your browser. Retrieve the initial administrator password on the instance:

```
sudo cat /var/lib/jenkins/secrets/initialAdminPassword
```

Enter the password, install the suggested plugins, create an administrator account, and finish the setup wizard. Confirm that the Jenkins dashboard loads.



4.3 Phase 3 — Install Git, Docker, and curl

Install Git, Docker Engine, and curl, then enable Docker at boot:

```
sudo dnf install -y git docker curl
git --version
sudo systemctl enable --now docker
```

```
sudo systemctl status docker
sudo docker --version
sudo docker ps
```

Allow the Jenkins service account to use Docker, then restart Jenkins so the new group membership takes effect:

```
sudo usermod -aG docker jenkins
sudo usermod -aG docker ec2-user
sudo systemctl restart jenkins
```

Reconnect your SSH session before testing Docker access as `ec2-user`.

> **Security warning:** Docker-group membership is effectively root-level access to the host. Use this configuration only in a disposable learning environment. For production, use an isolated build agent and a hardened container-build design.

4.4 Phase 4 — Prepare the source repository

The project repository is https://github.com/Yui096/ritesh-thorat-netflix-cicd.git. It should contain at least these files:

```
ritesh-thorat-netflix-cicd/
|-- index.html
`-- Dockerfile
```

Use the existing project files in that repository. A Dockerfile for a static Nginx-hosted site typically looks like this:

```
FROM nginx:alpine
COPY index.html /usr/share/nginx/html/index.html
EXPOSE 80
```

If you are initializing the repository locally for the first time, commit and push the files:

```
git init
git add index.html Dockerfile
git commit -m "Add Ritesh Thorat landing page"
git branch -M main
git remote add origin https://github.com/Yui096/ritesh-thorat-netflix-cicd.git
git push -u origin main
```

If `origin` is already configured, do not add it again. Confirm that `index.html` and `Dockerfile` appear on the GitHub repository page.

The repository is public in this configuration, so Jenkins can clone it without GitHub credentials. If the repository is private, configure a Jenkins Git credential instead.

4.5 Phase 5 — Create the Docker Hub repository and access token

1. In Docker Hub, create a repository named `ritesh-thorat-netflix-cicd` under your confirmed Docker Hub username.
2. Create a personal access token with the minimum permissions needed to push images.
3. Store the token securely. Never commit it to GitHub, place it in the Dockerfile, or hardcode it in the shell script.

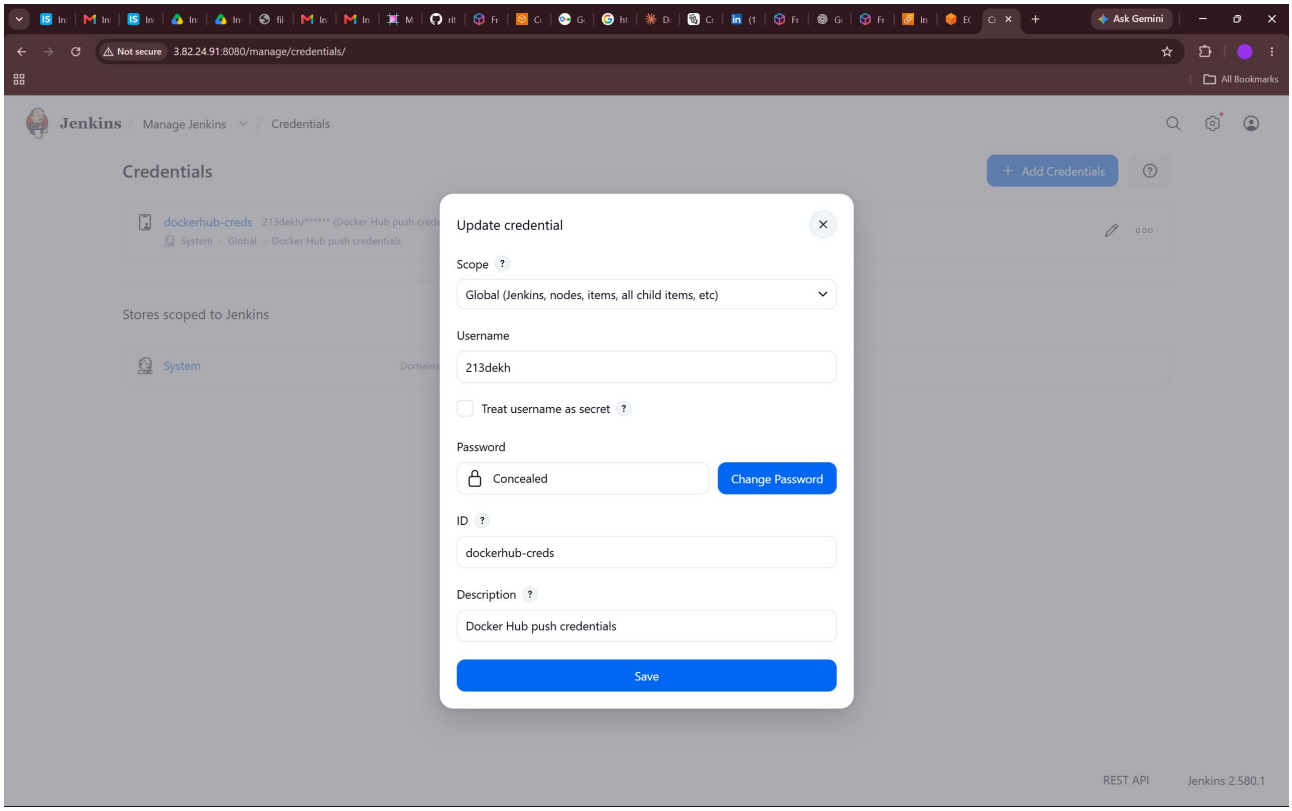
The image reference used by the job is:

```
<DOCKERHUB_USER>/ritesh-thorat-netflix-cicd:latest
```

4.6 Phase 6 — Configure Jenkins plugins and credentials

1. Go to **Manage Jenkins** → **Plugins**. Install or verify the **Git** and **Credentials Binding** plugins. Restart Jenkins if prompted.
2. Go to **Manage Jenkins** → **Credentials** → **System** → **Global credentials (unrestricted)**.
3. Add a **Username with password** credential:

Field	Value
Username	Your Docker Hub username
Password	Your Docker Hub access token
ID	`dockerhub-creds`
Description	Docker Hub push credentials



4.7 Phase 7 — Create the Jenkins Freestyle job

Create a new **Freestyle project**, for example `Ritesh-Thorat-CICD-Pipeline`.

Under **Source Code Management** → **Git**, configure:

Setting	Value
Repository URL	`https://github.com/Yui096/ritesh-thorat-netflix-cicd.git`

Setting	Value
Credentials	None (public repository)
Branch Specifier	`*/main`

Under **Build Environment**, enable the credentials binding option (shown as **Use secret text(s) or file(s)**). Add a **Username and password (separated)** binding and select `dockerhub-creds`:

Binding field	Environment variable
Username Variable	`DOCKERHUB_USER`
Password Variable	`DOCKERHUB_TOKEN`

Under **Build Steps**, select **Execute shell** and paste the script from Section 5. Save the job.

5. Build Script

Paste this script into the Jenkins **Execute shell** build step. It validates the source files, builds and tests the image, pushes it to Docker Hub using the bound credential variables, and deploys it on port 8081.

Item	Value
Image name	`${DOCKERHUB_USER}/ritesh-thorat-netflix-cicd:latest`
Temporary test container	`ritesh-thorat-test` (host `127.0.0.1:18081` → container 80)
Deployed container	`ritesh-thorat` (host 8081 → container 80)
Test URL	`http://127.0.0.1:18081/`
Verification text	`Stories worth staying up for`

```
#!/bin/bash
set -eu

# ========================================
# CONFIGURATION
# ========================================

: "${DOCKERHUB_USER:?Set DOCKERHUB_USER using Jenkins credentials binding}"
: "${DOCKERHUB_TOKEN:?Set DOCKERHUB_TOKEN using Jenkins credentials binding}"

IMAGE="${DOCKERHUB_USER}/ritesh-thorat-netflix-cicd:latest"

TEST_CONTAINER="ritesh-thorat-test"
APP_CONTAINER="ritesh-thorat"

TEST_PORT="18081"
APP_PORT="8081"

# ========================================
# CLEANUP FUNCTION
# ========================================

cleanup_test() {
    docker rm -f "$TEST_CONTAINER" >/dev/null 2>&1 || true
}
```

```
trap cleanup_test EXIT

# ========================================
# STEP 1: VERIFY SOURCE CODE
# ========================================

echo "========================================"
echo "STEP 1: Verify source code"
echo "========================================"

test -s index.html
test -s Dockerfile

echo "SUCCESS: Required source files found."

# ========================================
# STEP 2: BUILD DOCKER IMAGE
# ========================================

echo "========================================"
echo "STEP 2: Build Docker image"
echo "========================================"

docker build --pull -t "$IMAGE" .

echo "SUCCESS: Docker image built."

# ========================================
# STEP 3: TEST DOCKER IMAGE
# ========================================

echo "========================================"
echo "STEP 3: Test Docker image"
echo "========================================"

cleanup_test

docker run -d \
    --rm \
    --name "$TEST_CONTAINER" \
    -p "127.0.0.1:${TEST_PORT}:80" \
    "$IMAGE"

sleep 3

if curl -fsS \
    "http://127.0.0.1:${TEST_PORT}/" \
    -o /tmp/ritesh-thorat-test.html; then

    echo "SUCCESS: Website returned an HTTP success response."

    echo "Preview of returned HTML:"
    head -n 10 /tmp/ritesh-thorat-test.html

else
    echo "ERROR: Website HTTP test failed."
    docker logs "$TEST_CONTAINER" || true
    exit 1
fi

docker stop "$TEST_CONTAINER" >/dev/null

trap - EXIT
```

```
echo "SUCCESS: Docker image test passed."

# ========================================
# STEP 4: LOGIN TO DOCKER HUB
# ========================================

echo "========================================"
echo "STEP 4: Login to Docker Hub"
echo "========================================"

printf '%s' "$DOCKERHUB_TOKEN" |
    docker login \
        --username "$DOCKERHUB_USER" \
        --password-stdin

echo "SUCCESS: Docker Hub authentication completed."

# ========================================
# STEP 5: PUSH IMAGE TO DOCKER HUB
# ========================================

echo "========================================"
echo "STEP 5: Push image to Docker Hub"
echo "========================================"

docker push "$IMAGE"

echo "SUCCESS: Docker image pushed."

# ========================================
# STEP 6: DEPLOY APPLICATION
# ========================================

echo "========================================"
echo "STEP 6: Deploy application on EC2"
echo "========================================"

# Remove the previous application container.
docker rm -f "$APP_CONTAINER" >/dev/null 2>&1 || true

# Run the updated application container.
docker run -d \
    --name "$APP_CONTAINER" \
    --restart unless-stopped \
    -p "${APP_PORT}:80" \
    "$IMAGE"

echo "SUCCESS: Application container started."

# ========================================
# STEP 7: VERIFY DEPLOYMENT
# ========================================

echo "========================================"
echo "STEP 7: Verify deployment"
echo "========================================"

sleep 3

if curl -fsS \
    "http://127.0.0.1:${APP_PORT}/" \
    -o /tmp/ritesh-thorat-deployed.html; then

    echo "SUCCESS: Deployed website returned an HTTP success response."
```

```
    echo "Preview of deployed HTML:"
    head -n 10 /tmp/ritesh-thorat-deployed.html

else
    echo "ERROR: Deployment verification failed."
    docker logs "$APP_CONTAINER" || true
    exit 1
fi

# ========================================
# STEP 8: LOGOUT FROM DOCKER HUB
# ========================================

echo "========================================"
echo "STEP 8: Logout from Docker Hub"
echo "========================================"

docker logout || true

# ========================================
# FINAL STATUS
# ========================================

echo "========================================"
echo "CI/CD PIPELINE COMPLETED SUCCESSFULLY"
echo "========================================"

docker ps --filter "name=$APP_CONTAINER"

echo "Container name: $APP_CONTAINER"
echo "Container port: 80"
echo "EC2 host port: ${APP_PORT}"
echo "Website URL: http://<EC2_PUBLIC_IP>:${APP_PORT}"
```

Script notes:

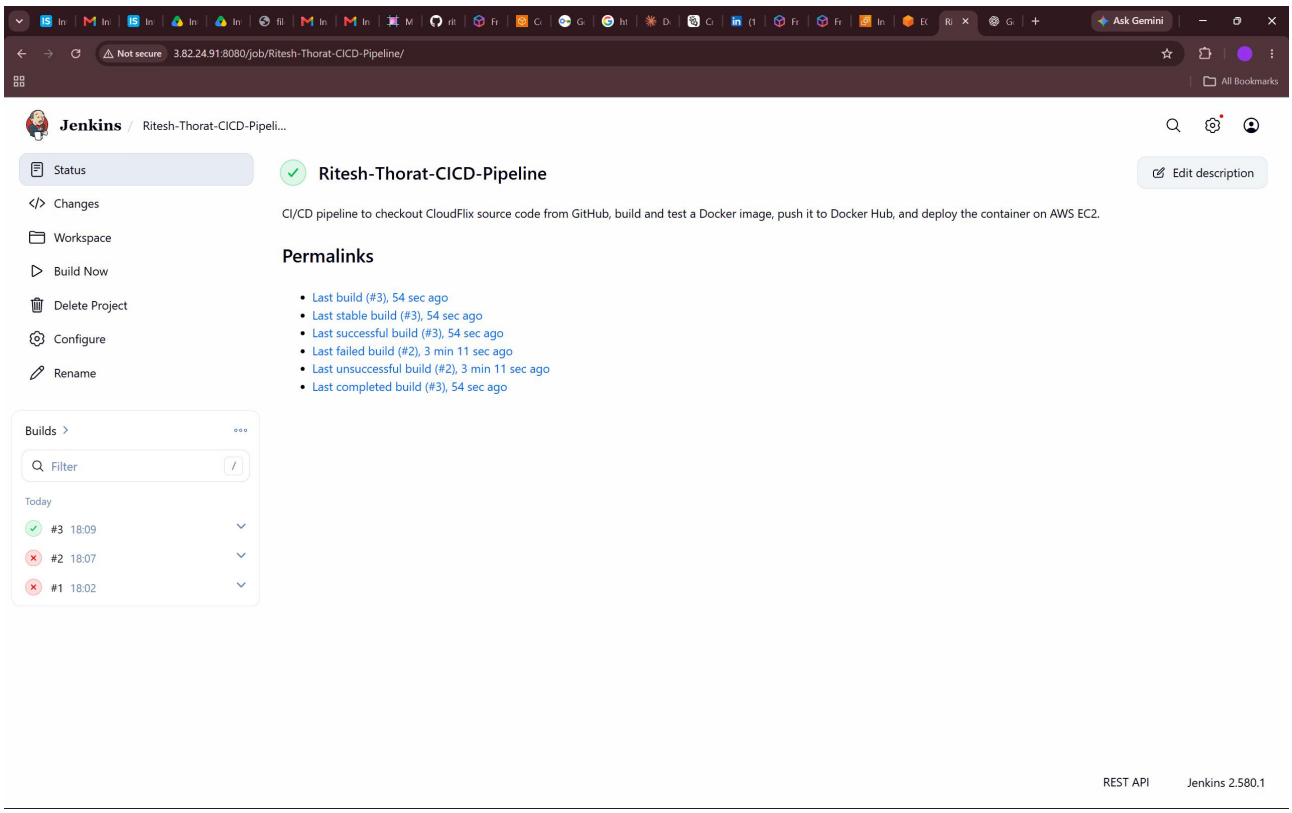
- The test phrase `Stories worth staying up for` comes from the supplied project infographic. If your `index.html` uses different text, replace it with a distinctive phrase that exists in your page.
- The test container binds only to `127.0.0.1:18081`, so it is not exposed to the network and does not conflict with the deployed container on 8081.
- The script builds the image locally on EC2, pushes it to Docker Hub, and starts the deployed container from the local image on the same host.
- The token is passed through standard input (`--password-stdin`) and is never printed.

6. Verification

6.1 Run the Jenkins job

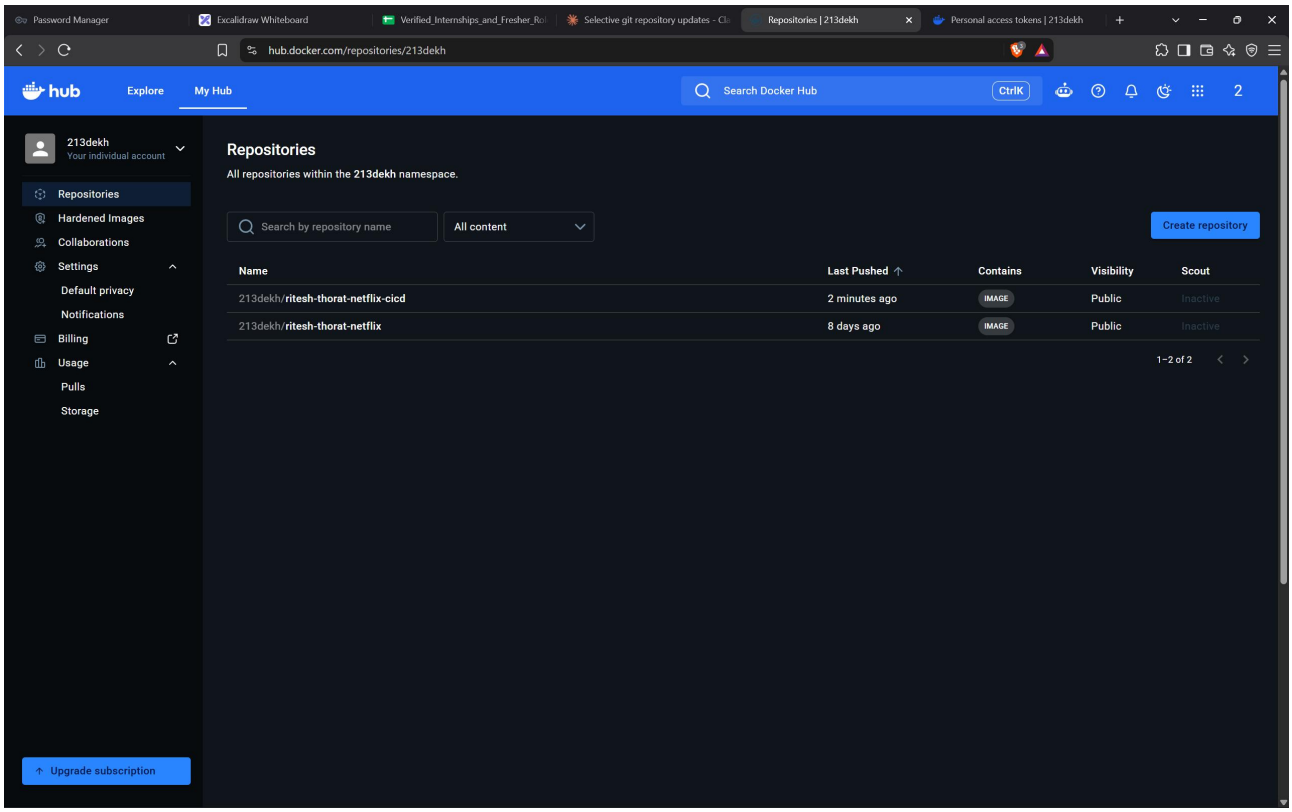
Open `Ritesh-Thorat-CICD-Pipeline` and select **Build Now**. When the build finishes, open the build number and select **Console Output**. A successful run shows the step messages and ends with:

```
CI/CD PIPELINE COMPLETED
...
Finished: SUCCESS
```



6.2 Verify the Docker Hub image

Open the repository under your Docker Hub username and confirm that the `latest` tag exists for `ritesh-thorat-netflix-cicd`.

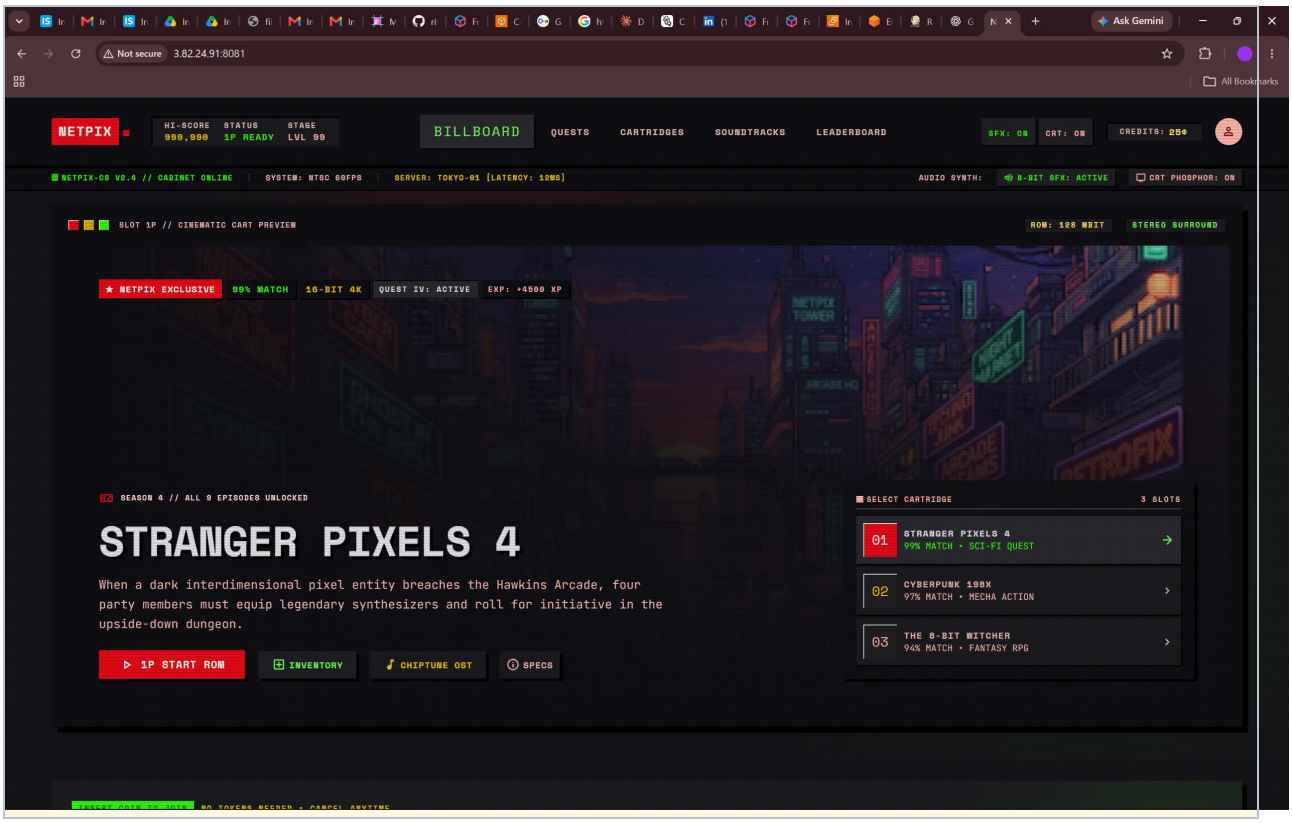


6.3 Verify the live website

Open this URL in a browser, replacing the placeholder with your EC2 public IPv4 address:

```
http://YOUR_EC2_PUBLIC_IP:8081
```

Confirm that the landing page renders. Docker serves the page by mapping host port 8081 to Nginx container port 80.



6.4 Optional continuous-integration test

The first pipeline run is manual. To demonstrate a source change, edit `index.html`, then commit and push:

```
git add index.html
git commit -m "Update Ritesh Thorat landing page"
git push origin main
```

Run the Jenkins job again. Confirm that it checks out the new commit, rebuilds and pushes the image, and replaces the running container. To build automatically on every push, configure a GitHub webhook and the matching Jenkins build trigger.

7. Troubleshooting

Symptom	Checks
Browser cannot reach the site	Confirm the security group allows TCP 8081 from your current public IP. Then run the Docker commands below.
Jenkins cannot run Docker commands	Run `id jenkins` to confirm Docker-group membership, then restart Jenkins.

Symptom	Checks
Docker Hub push fails	Confirm that `DOCKERHUB_USER` is the real Docker Hub namespace, the repository exists under that account, the token has push permission, and `dockerhub-creds` is bound to the expected variables.
Verification step fails	Confirm that the `grep` phrase exists in `index.html`.

Inspect the Docker service and the deployed container:

```
sudo systemctl status docker
sudo docker ps
sudo docker logs ritesh-thorat
curl -I http://localhost:8081
```

Check Jenkins and Docker-group membership:

```
id jenkins
sudo systemctl restart jenkins
sudo systemctl status jenkins
```

> **Reminder:** Never print the Docker Hub token in console output, and never commit credentials to source control.